

Induction Practical (Practical 1): Eclipse Project Setup

Introduction and Learning Outcomes

To gain proficiency with Eclipse for configuring projects tailored to run on specific FPGA development boards, you'll delve into its disassembly capabilities as well. This journey encompasses:

- Understand the full potential of Eclipse as your Integrated Development Environment (IDE), essential for efficiently managing FPGA and RTOS projects
- Uncover the intricacies of disassembling simple assembly programs within Eclipse
- Leverage Eclipse's disassembly window to update and refine assembly programs directly

Practical guidelines

You should be preparing and attempting to complete all tasks for this practical at home, using your DE10 board. Come to the practical ready to ask questions and complete the post-practical quiz to consolidate your knowledge. This practical will run over 1 week - [see full schedule here](#).

This practical activity should be completed individually. You can and should discuss the questions and concepts with other students. However, you should develop and write your own code initially, before sharing ideas and code with other students. Copying other students' work and presenting it as your own is not useful in the preparation for assessments that require the knowledge of the practical.

You can also use generative AI to assist you with the code and troubleshooting process but it is recommended to use it in a limited capacity as long as you still understand the coding and concepts required in this practical activity. During assessments, you will need to explain your understanding and code to the demonstrators without the assistance of generative AI.

Please inform a demonstrator about your progress before leaving the practical session.

Key Information and Notices:

Revision History

This is a live document so we recommend not to print this. Use this revision history to monitor for any changes we may need to make to this document to clarify requirements or provide hints.

Description	Version	Date
Initial release	v1.0.0	21/02
Removed mention of getting_started.s section since this heading was removed. Added link to section 2 of help manual to install patch for header files under section 2.1 of practical 1	v1.0.1	02/03

Preparation for this Practical

Before attempting any prac activity, read the entire prac document and complete the suggested preliminary prac quiz on Moodle.

On-campus Equipment Requirements per Group

- DE10 Lite board or a DE2-115 board
- Laptop (with Quartus installed) or Lab PC
- Nios II Software Build Tools for Eclipse (Quartus Prime 18.1)

Board	Family	Device Name
DE10 (take-home boards)	Max 10	10M50DAF484C7G
DE2 (on-campus boards)	Cyclone IV	EP4CE115F29C7

Estimated Time Commitment: 3 hours per student (note: this is only an estimate of how long the prac will take you)

Part 1 - Preliminary Work

Assume a 16-bit number is stored in register `r4`. Using the [Instruction Set Reference](#), write Nios II assembler instructions that will modify the bits stored in Register `r4` that will:

- Invert bits 1 and 0
- Set (turn on) bits 5 and 4
- Reset (turn off) bits 7 and 6

All other bits should be left unchanged. Finish off this piece of code with a jump immediate to address `0x0000002B0`:

```
jmpi 0x0000002B0
```

Write down the instructions and then convert these into machine code using the instruction set reference. These will be used for Exercise 3. You can find valuable information in the feedback for each question in the preliminary practical 1 quiz.

Practical Activities:

Part 2 - Running your first program

Before starting, make sure to follow “[1.2.0 Licensing](#)” to set up the Monash licence. You will need to use the Monash VPN to access the licence.

2.1 Project Setup

In this first part, you will use the *Nios II Software Build Tools for Eclipse (Quartus Prime 18.1)* to download a pre-written hardware design and assembler program onto the DE-board. To prepare the workspace do the following:

1. Download the [prac1.zip](#) file from Moodle. This will contain the .sof file and .sopcinfo files required to program the DE-board.
2. Create a new folder for the files required for this exercise. Make sure the folder name is unique and the path **does not contain any spaces**. If you are using lab computers, don't save files on the desktop folder as it is a network mapped folder. Try "C:\Temp". (Please note that this folder gets cleaned on a daily basis)
3. Start the *Nios II Software Build Tools for Eclipse (Quartus Prime 18.1)* either using the desktop shortcut or by searching for it in the windows start menu. From now on, we will refer *Nios II Software Build Tools for Eclipse (Quartus Prime 18.1)* as “Eclipse”
4. When you open Eclipse, it will ask for a workspace, choose the folder you created for this project.

Now we will start programming the DE-10 or DE-2 board by doing the following:

1. In the Eclipse program, open Quartus Prime Programmer with “Nios II > Quartus Prime Programmer”.
2. Once the Programmer window shows up, confirm that the hardware is detected in hardware setup. Click on **Add File ...** and navigate to **prac1** folder and look for your correct hardware i.e. if you use DE10 board, the file would be: **.\prac1\hardware\DE10\DE10_Lite_Computer.sof**.
3. Click on the chip image and press **Start**, the board configuration will be downloaded to the physical device. Now the DE2 (or DE10) board should be configured as a Nios II microprocessor system. (If the **Start** button is greyed out, click on hardware setup and in the drop-down menu “Currently selected hardware” choose USB-blaster [USB-0])

If this is successful, you should see “100% successful” highlighted in the top right “progress” section.

2.2 Running the program

1. Create a new project with “File > New > Nios II Application and BSP from Template”
2. In this project enter the details like in the figure below:
 - a. Within “Target hardware information > SOPC Information file name:” Click the “...”, and choose the .sopcinfo file for the board you are programming. It should be in “./hardware/DE2_115/” or “./hardware/DE10/”
 - b. Name this project prac1

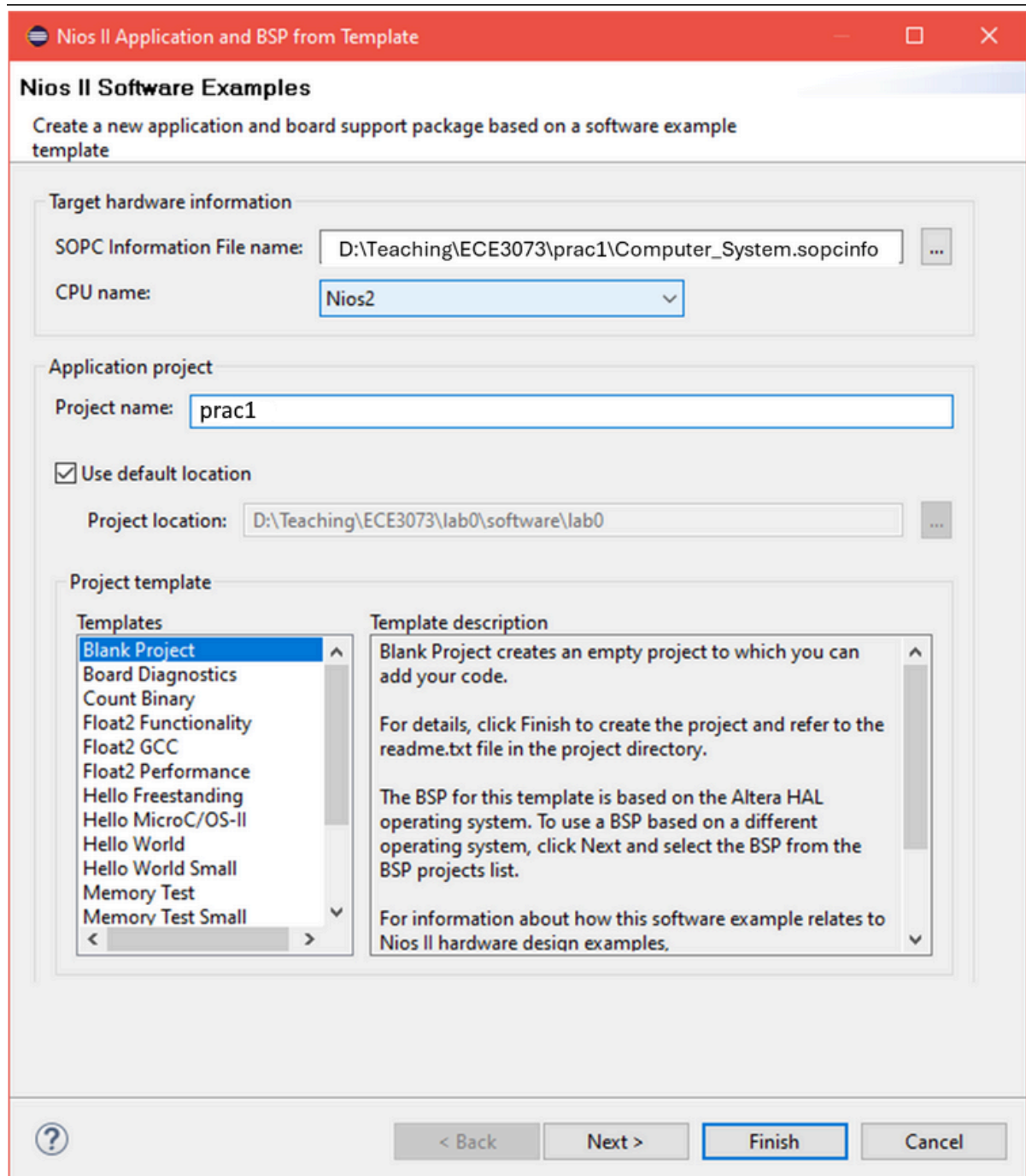


Figure 1: Nios II application from template setup window

3. Right-click on “prac1” inside the project explorer.
 - a. Choose “New > file” and ensure prac1 is the parent folder
 - b. Name the file “getting_started.s”

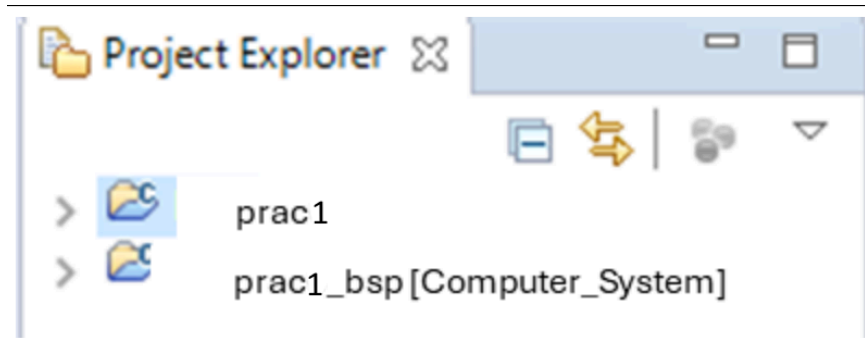


Figure 2: Project Explorer Tab

4. Copy the code within the “getting_started.s” file into the “getting_started.s” file.
5. Repeat Step 3 but name the file “address_map_nios2.S”
 - a. Copy the contents of the file with the same name into this file
6. Click on the “Build All Icon, , or press “Ctrl + B”.

Note: If you the project refuses to be created, please refer to [chapter 2 of the help manual](#) to install some patches for the application.

7. Download the compiled file to the board by selecting “Run > Run Configurations”. Double-click Nios II Hardware. See Figure 3 as an example of what you will see.

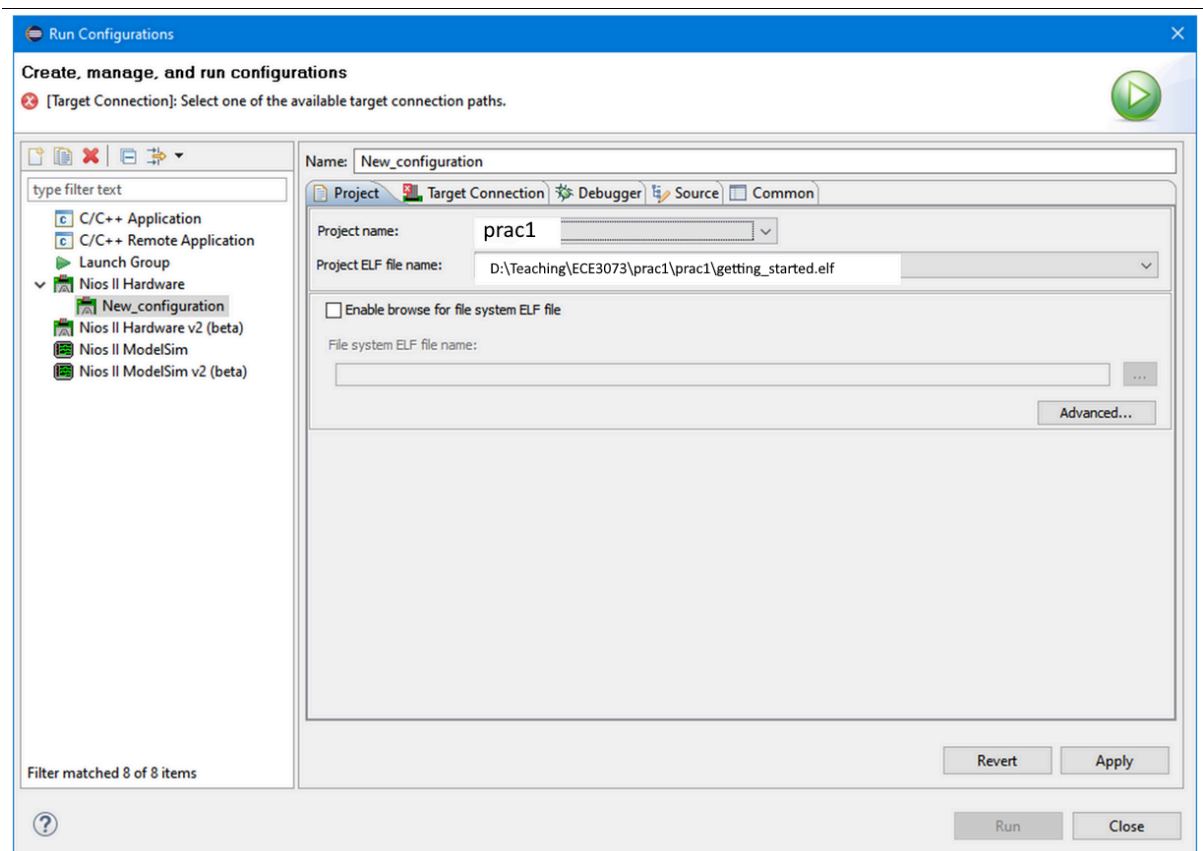


Figure 3: Run configurations

- a. Next, cycle to the “Target Connections” tab. Applying the following settings
 - i. Tick “Ignore mismatched system ID” and “Ignore mismatched system timestamp” shown in Figure 4. Then click “Refresh Connection” to display the device. The downloaded hardware configuration will contain two different options, either option will work.
 - ii. Click “Apply”, Then “Run”

Note: If you get an issue with “File not found “C:\intelFPGA.....”, you can make a copy of the file that is causing the issue by copying the non “RGB” version within the same directory. Call a demonstrator for additional assistance if required.

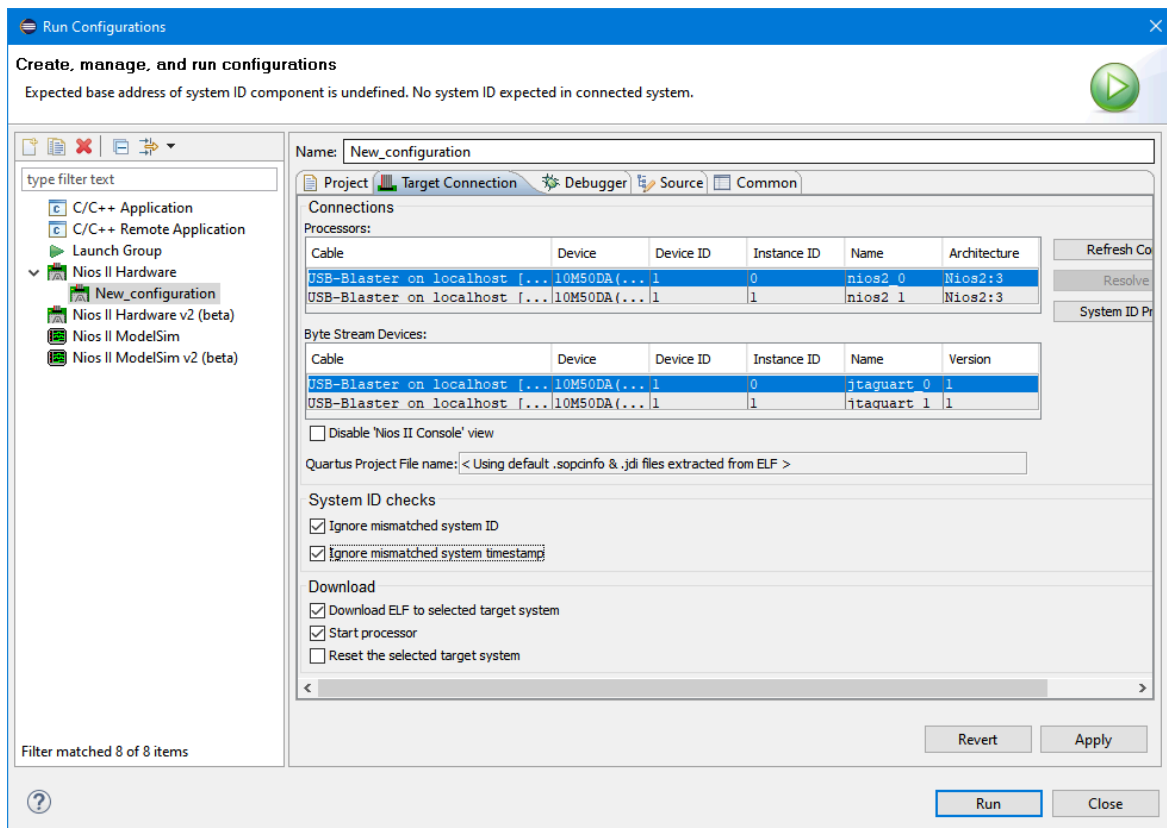


Figure 4: Target Connection Tab

8. Confirm that the program does the following:
 - Displays the DE2 (or DE10) board's SW switch states on the red lights (LEDR)
 - Shows a rolling LEDR display. If any of the KEYs are pressed, the pattern is changed to correspond to the settings of the SW switches.
 - o Try setting the first 2 bits of the SW high and pressing the KEYs. What happens?

For DE10 boards: These only have 10 switches, so the value is referred to as "SW[9:0]". Also, note that the behaviour of the SW to LEDR is slightly different. The DE10 sample program rolls the values in the 8 left-most switches.

In *Nios II Software Build Tools for Eclipse (Quartus Prime 18.1)*, there is no way to stop the assembly code once you run it. To run a different program you can simply load the new software and the current program will be overwritten. You can check if you have completed this correctly with a demonstrator, otherwise, you can move on to the next part.

Template code for getting_started.s and address_map_nios2.s can be found [here](#).

Part 3 - Debugging Assembly

3.1 Debug Setup

In this second exercise, we want to have a closer look at the assembly code under the hood. We will achieve that through the debugger that comes with *Nios II Software Build Tools for Eclipse* (Quartus Prime 18.1).

To debug the assembly file:

1. Open the file “getting_started.s” file in the file explorer menu.

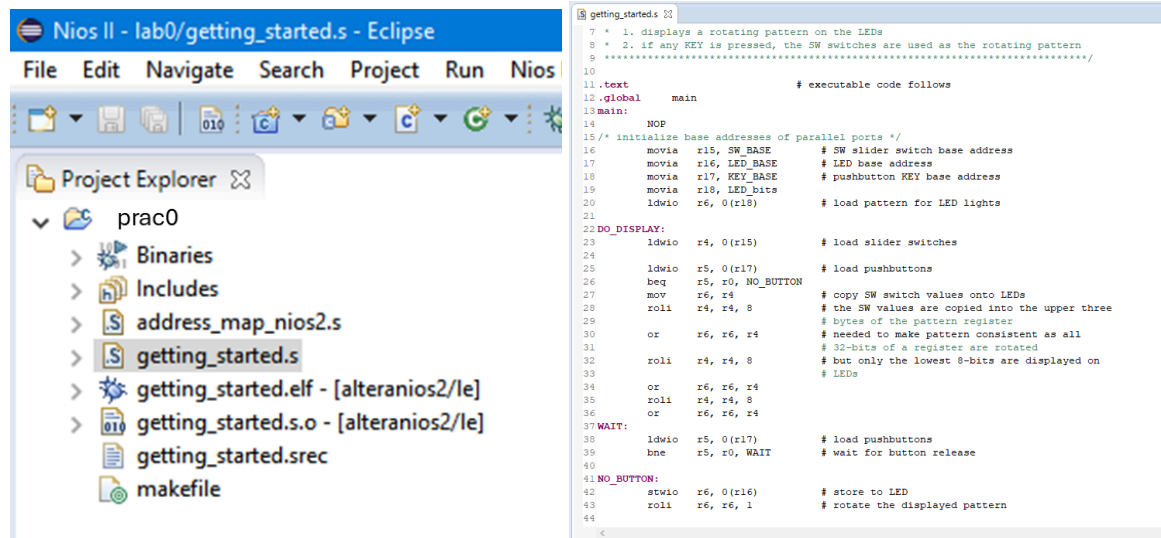


Figure 5: Opening the assembly code

2. Place a breakpoint on line 14 (by double-clicking), you will see a breakpoint inserted like so:

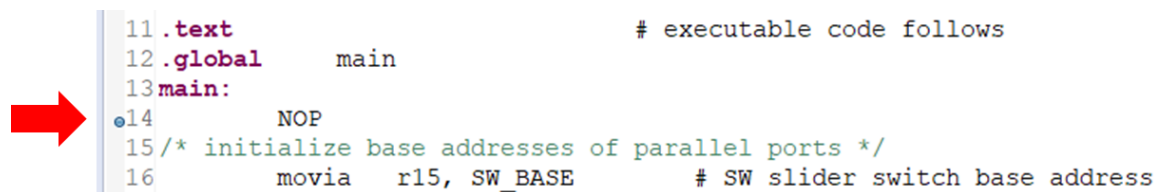


Figure 6: Inserting a breakpoint at line 14.

3. Go to “Run > Debug Configuration”. Click **Debug** in the window shown:

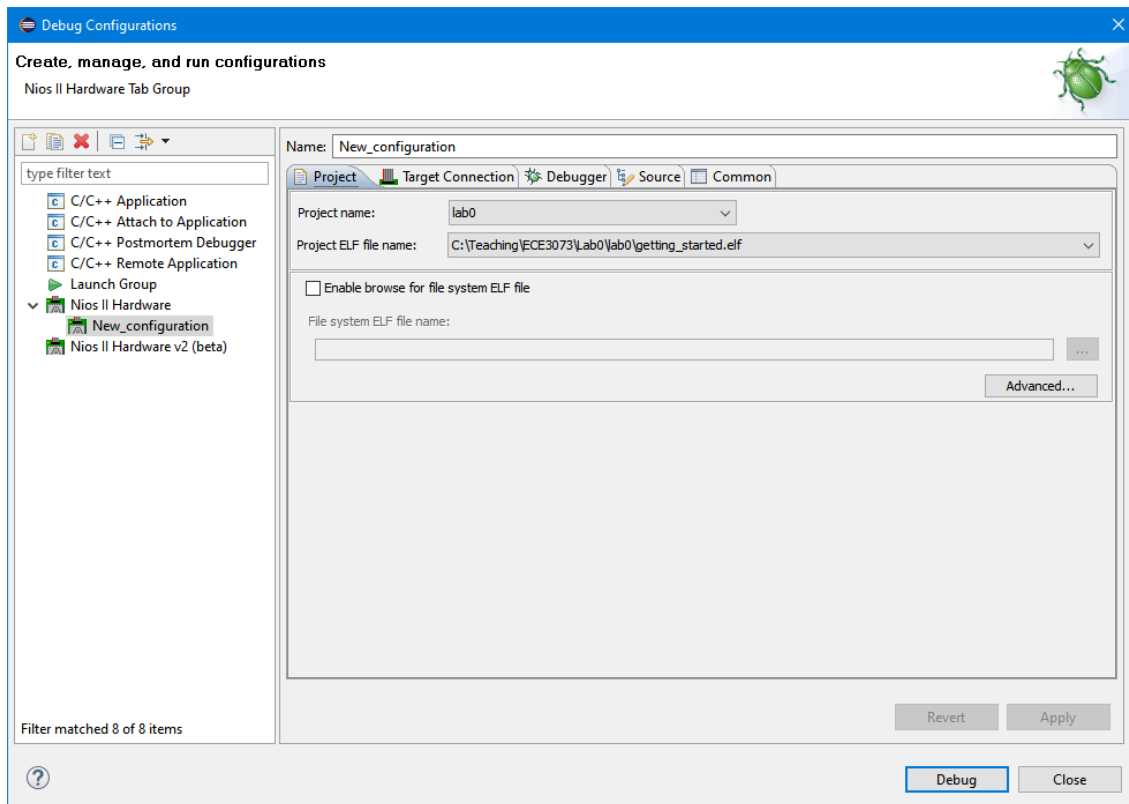


Figure 7: Debug Window

A window will pop up asking if you want to change to Debug Perspective. Please choose “Yes”.

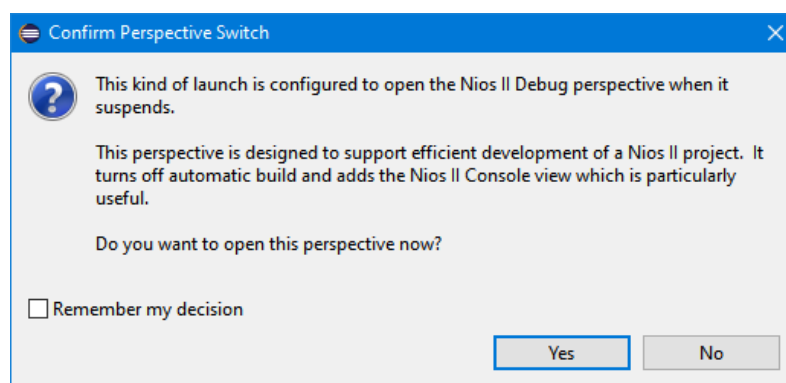


Figure 8: Perspective switch window

You should now see something like so:

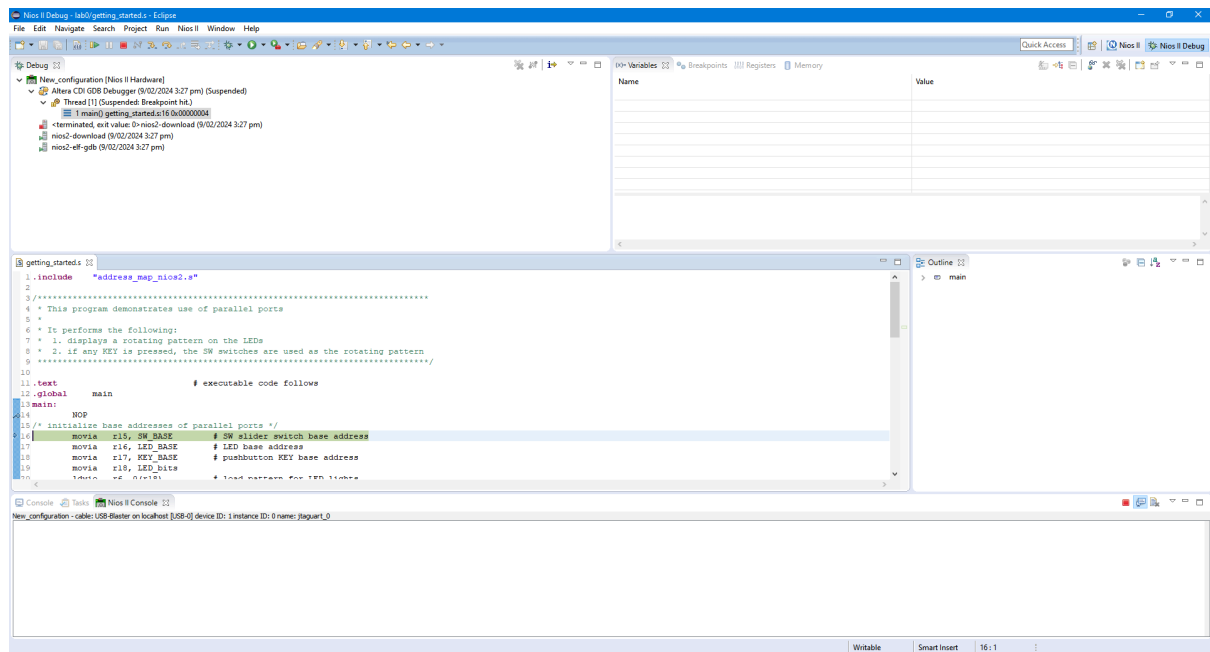


Figure 9: Debug Perspective

We will dive deeper into breakpoints and the debugger in [section 3.3](#).

3.2 Using the Debugger

Using the Disassembly Window:

1. Pseudo operations are instructions where the original source code instruction is not the same as the disassembled instruction/s as displayed in the Disassembly Window. Use this fact (or refer to the Nios instruction reference on Moodle) to identify all of the pseudo operations in “getting_started.s” and highlight the actual code - you may choose to use a word editor for this.
2. To view the Disassembly window, Go to “Windows > Show View > Other...” Then choose “Debug > Disassembly”. Then you can drag the Disassembly window so that it can be next to other locations such as this:

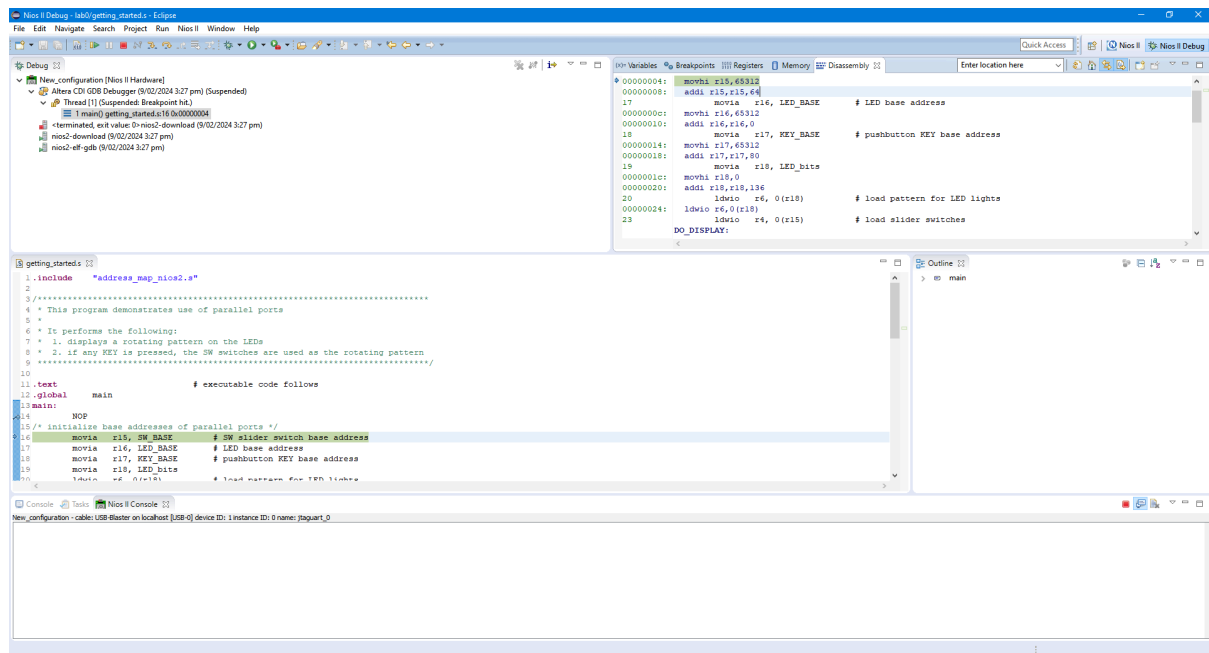


Figure 10: Added Disassembly viewer

Single Stepping is a key tool for debugging as it allows you to see what the code is doing line-by-line and diagnose the specific point at which errors occur. To do so:

1. Single step the program up to the delay loop (you can do that with step into or step over highlighted in Figure 10):

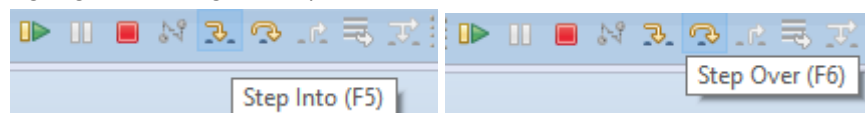


Figure 11: Debugger options with shortcuts

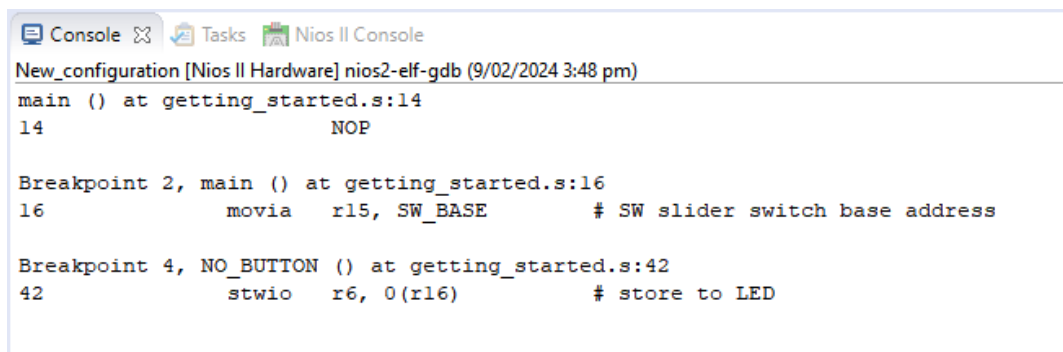
2. Before you single step the program you should anticipate what the result of the next instruction will be in terms of changes to registers including the program counter, changes to memory, lights on the DE2 (or DE10) board, etc. After each single step check that you anticipated correctly.

(By default, the Register view is displayed in decimal, you can 'Right Click > Format' any register and change it to Hexadecimal.)

3.3 Deeper dive into the debugger

Using Breakpoints – An instruction breakpoint provides the means to stop a Nios II program when it reaches a specific address in the assembly code. A simple procedure for setting an instruction breakpoint is:

1. In the Disassembly window, scroll to display the instruction address that will have the breakpoint. As an example, scroll to the instruction at the label `DO_DISPLAY`, which is address `0x000002AC`. In a larger program, it may be easier to enter the address directly. You can do this by entering the hexadecimal address into the “Enter Location Here” option. For example, we could have entered “0x58” to jump to the same instruction as before.
2. Double-click on the space left of the address `0x000002AC` in the Disassembly window. The window displays a blue dot next to the address to show that an instruction breakpoint has been set. Double-clicking the same location again removes the breakpoint.
3. Once the instruction breakpoint has been set, run the program. The breakpoint will trigger when the pc register value equals `0x000002AC`. Control then returns to the Debugger, and the Disassembly window highlights in green the instruction at the breakpoint. A corresponding message is shown in the Console pane:



```
Console Tasks Nios II Console
New_configuration [Nios II Hardware] nios2-elf-gdb (9/02/2024 3:48 pm)
main () at getting_started.s:14
14      NOP

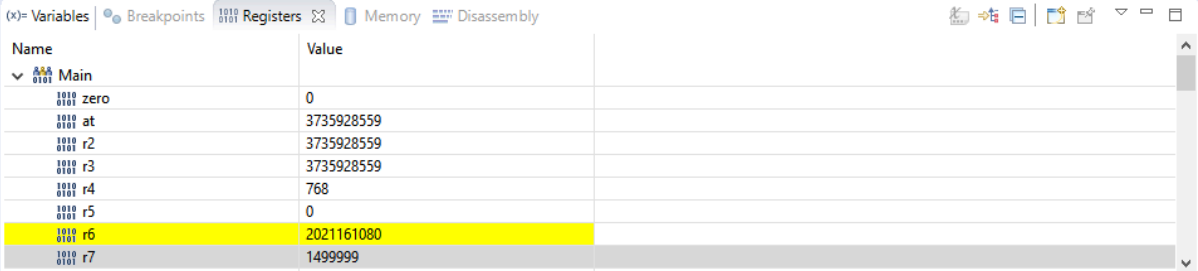
Breakpoint 2, main () at getting_started.s:16
16      movia    r15, SW_BASE      # SW slider switch base address

Breakpoint 4, NO_BUTTON () at getting_started.s:42
42      stwio    r6, 0(r16)        # store to LED
```

Figure 12: Console pane after stopping at a breakpoint

3.3.1 Examining and changing register contents

1. The Registers tab, displayed in Figure 12, shows the value of each register in the Nios II processor and allows the user to edit most of the register values. The number format of the register values can be changed by right-clicking each register and selecting “Change Value...”. Each time program execution is halted, the Debugger updates the register values. If you select another register, the register you just updated will be highlighted in yellow. The user can also edit the register values while the program is halted. Any edits made are visible to the Nios II processor when the program’s execution is resumed.



Name	Value
▼ Main	
zero	0
at	3735928559
r2	3735928559
r3	3735928559
r4	768
r5	0
r6	2021161080
r7	1499999

Figure 13: Registers Tab

2. As an example of editing a register value, first scroll the Disassembly window to the label DELAY, which is at address `0x000002EC`. Set a breakpoint at this address and then run the program. After the breakpoint triggers and control returns to the Register tab, notice that there is a large value in register `r7`. This value is used as a counter in the delay loop (it will decrement at every iteration). Single Click (or “Right Click > Change Value...”) on the contents of register `r7` and edit it to the value `1`. Press Enter on the computer keyboard, or click away from the register value to apply the edit. Now, single-step the program to see that it exits from the delay loop after one more iteration when `r7` becomes `0`.

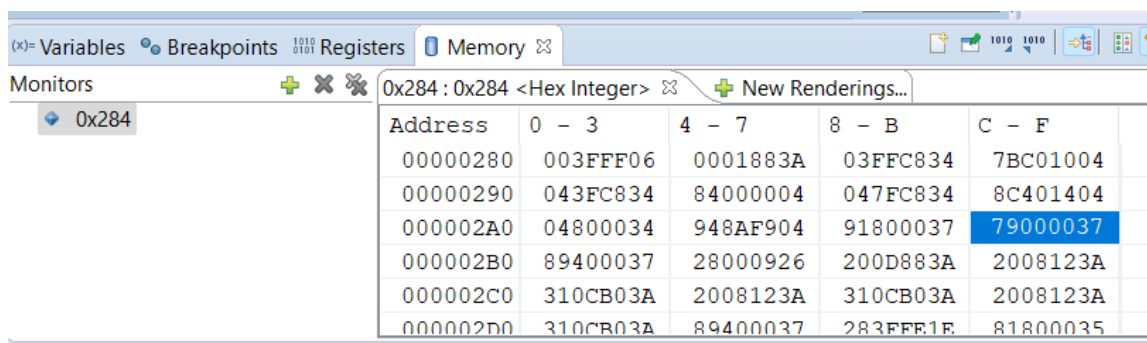
Feel free to approach a TA in your practical session if you wish to discuss them further:

Part 4 - Modifying Machine Code in the Debugger

In this last exercise, you are asked to modify the assembler code in “getting_started.s” and use the functions available in the Debugger to test your modifications. The changes to the program will be made by directly adding NIOS machine code instructions to the program memory.

As it originally stands, “getting_started.s” reads the state of the slider switches on the DE2 (or DE10) board and then immediately writes those values to the red LEDs. The changes that you will make to the program will modify the data read from the slider switches before writing it to the LEDs. These modifications are described in the preliminary work section. You should already have machine code instructions for performing these operations in answer to the preliminary work (Part 1).

To view the memory, go to the memory tab, click on the Green Plus (Add Memory Monitor). Type `0x284` to the field, then click “OK”. A view of the memory block neighbouring address `0x284` will show up. Click on “New Renderings...” and Choose “Hex Integer”. “Hex Integer” will display the memory content in hexadecimal which is consistent with how we display these values in the workshops and preliminary work. You should see something like this



Address	0 - 3	4 - 7	8 - B	C - F
00000280	003FFF06	0001883A	03FFC834	7BC01004
00000290	043FC834	84000004	047FC834	8C401404
000002A0	04800034	948AF904	91800037	79000037
000002B0	89400037	28000926	200D883A	2008123A
000002C0	310CB03A	2008123A	310CB03A	2008123A
000002D0	310CB03A	89400037	283FFE1E	81800035

Figure 14: Memory Monitor

All of the required changes are as follows:

1. Overwrite the `ldwio r4,0(r15)` instruction at label `DO_DISPLAY`: with an unconditional branch “`br`” to memory address `0x000002F8`. Of course, you can work out the corresponding machine code to practice this however the machine code is given below. As well, the changed memory address is shown in Figure 14 below. Make sure to note down the machine code value that you replaced as you will use it later.

(Note: to change the machine code at this location you will need to change in the Memory tab instead of the Disassembly tab. Record your machine code for `ldwio r4,0(r15)` and branch instruction here: `ldwio r4,0(r15):79000037,br 0x2F8:00001206`)

Address	0 - 3	4 - 7	8 - B	C - F
00000270	10800104	10FFFD36	00003740	00003F80
00000280	003FFF06	0001883A	03FFC834	7BC01004
00000290	043FC834	84000004	047FC834	8C401404
000002A0	04800034	948AF904	91800037	00001206
000002B0	89400037	28000926	200D883A	2008123A
000002C0	310CB03A	2008123A	310CB03A	2008123A

Figure 15: Changed machine code at 0x2AC

- Write the machine code for the `ldwio r4, 0(r15)` instruction that you overwrote in part (1) at memory location `0x0000002F8`.
- Then in the following successive memory locations (groups of 4 bytes) insert the 4 machine code instructions **from your preliminary work**. You should see something like Figure 16.

Address	0 - 3	4 - 7	8 - B	C - F
00000270	10800104	10FFFD36	00003740	00003F80
00000280	003FFF06	0001883A	03FFC834	7BC01004
00000290	043FC834	84000004	047FC834	8C401404
000002A0	04800034	948AF904	91800037	00001206
000002B0	89400037	28000926	200D883A	2008123A
000002C0	310CB03A	2008123A	310CB03A	2008123A
000002D0	310CB03A	89400037	283FFE1E	81800035
000002E0	300C107A	01C005F4	39F8D804	39FFFFC4
000002F0	383FFE1E	003FED06	79000037	210000DC
00000300	21000C14	213FCFCC	00002B01	DEFFFC04
00000310	DF000315	DF000304	E13FFD15	E17FFE15

Figure 16: Further changes to machine code

Note that the Disassembly window will not immediately reflect the changes that you have made to memory. To disassemble your added code, click on the refresh view on the top right corner of the tab and you should see changes in the disassembly after the “Extra” label similar to those in Figure 17 below.

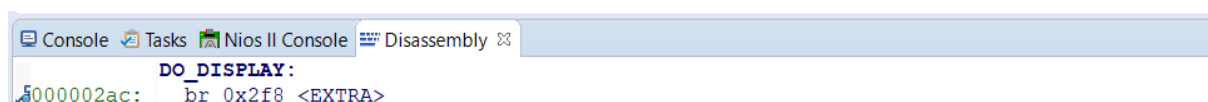


Figure 17: Refreshing Disassembly View

	EXTRA:			EXTRA:
000002f8:	nop		000002f8:	ldwio r4,0(r15)
54		nop	54	nop
000002fc:	nop		000002fc:	xori r4,r4,3
55		nop	55	nop
00000300:	nop		00000300:	ori r4,r4,48
56		nop	56	nop
00000304:	nop		00000304:	andi r4,r4,65343
57		nop	57	nop
00000308:	nop		00000308:	jmpj 0x2ac <DO_DISPLAY>
				alt_load_section:

Figure 18: Changes after refreshing disassembly window

Single step through your code to ensure that it works correctly before pressing “Resume”. Try setting the first 2 bits of the SW high and pressing the KEYS. What happens? Feel free to approach a TA in your practical session if you wish to discuss them further:

Please inform a demonstrator about your progress before leaving the practical session.